ZERO TO EBPF

eBPF

CLOUD-NATIVE KERNEL PROGRAMMING MADE SIMPLE

SANDIP KR. DEY

Forward

In recent years, the rise of **eBPF** and **Cilium** has redefined what's possible in Linux systems, networking, and security. But despite its growing impact, these technologies remain intimidating for newcomers. This ebook is my humble attempt to make that learning curve less steep by distilling the ecosystem into something clear, structured, and human.

This guide is not meant to be definitive. It's a living effort, built from community insights, late-night debug sessions, open-source rabbit holes, and personal curiosity. If you're reading this, you're already part of that story.

Please share your feedback, ideas, corrections, or resources you think should be included.

Your contributions, large or small, can help the next developer, student, or operator feel a little less lost and a little more empowered.

Let's keep building. Together.

Sincerely,
Sandip Kumar Dey
sandipkumardey615@gmail.com
GitHub (sandipkumardey)

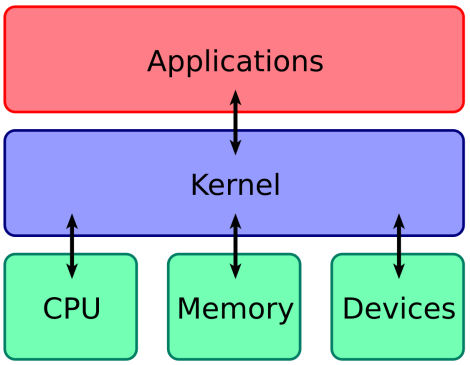
Chapter 1

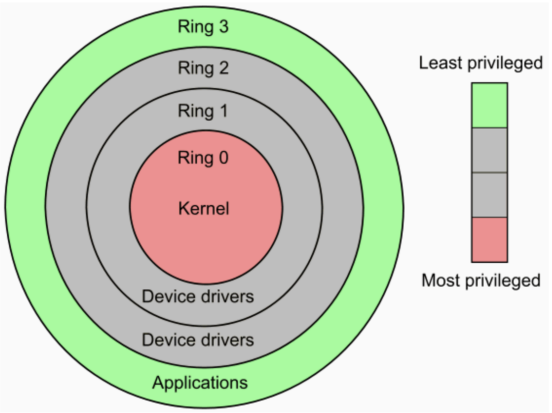
The Kernel: Still the Unsung Hero

What is a Kernel? (User Mode vs Kernel Mode)

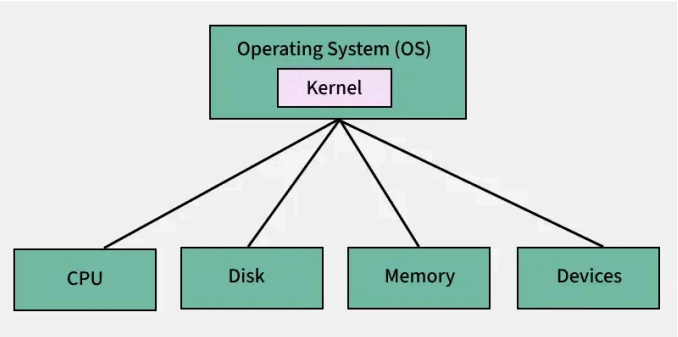
The kernel is the most critical software component of any operating system, functioning like the brain or central command center of a computer. It resides at the lowest level of the software stack and acts as an intermediary between applications (software) and physical components (hardware). Unlike regular applications that run in user mode—a restricted space with limited access to hardware—the kernel runs in kernel mode, where it has complete and privileged control over the CPU, memory, I/O devices, and system resources.

When an application, like Chrome or Spotify, wants to read a file, allocate memory, or access the network, it can't communicate with hardware directly. Instead, it sends requests to the kernel through system calls such as open, read, or write. The kernel ensures that these operations are securely and efficiently executed, managing hardware details on behalf of the software. The kernel is invisible to most users, yet it plays a foundational role in system security, stability, and performance.

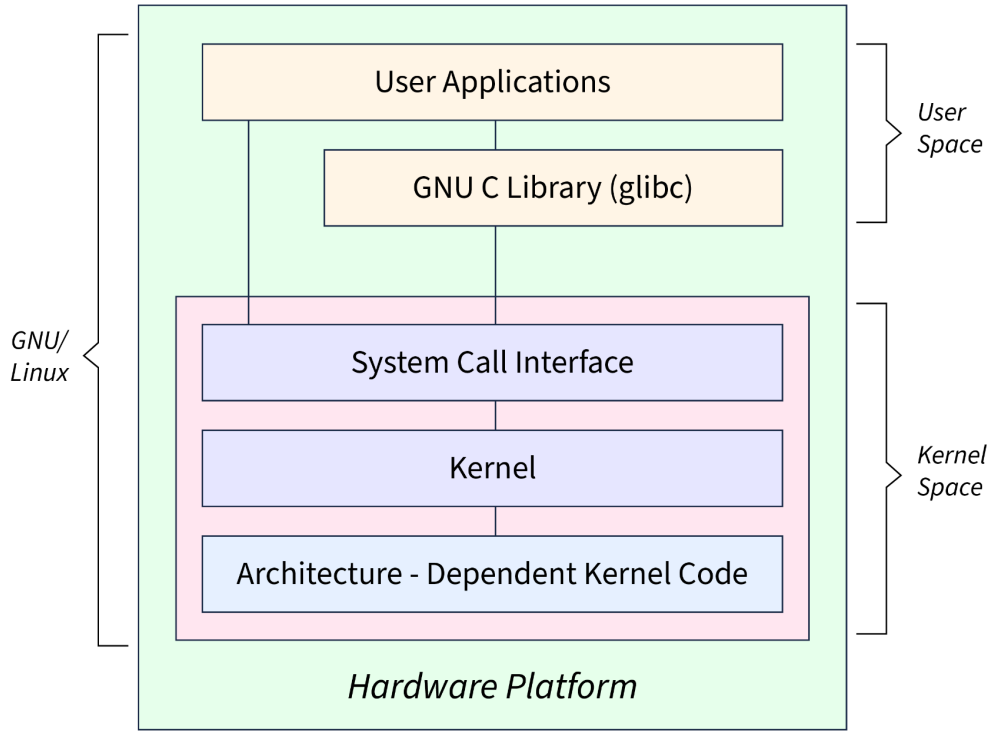




4 Core Jobs of a Kernel



Fundamental Architecture of Linux



SCALER Topics

The kernel has four major responsibilities that keep a computer functional and responsive:

1. CPU Scheduling (Time-Slicing): The kernel acts like a traffic cop, directing which program runs on the CPU and for how long. This is achieved through a

process called time-slicing, where the CPU rapidly switches between active programs, allowing multitasking. Whether you're coding in VS Code, streaming music, or watching a video, the kernel ensures that each task gets CPU time without interfering with others.

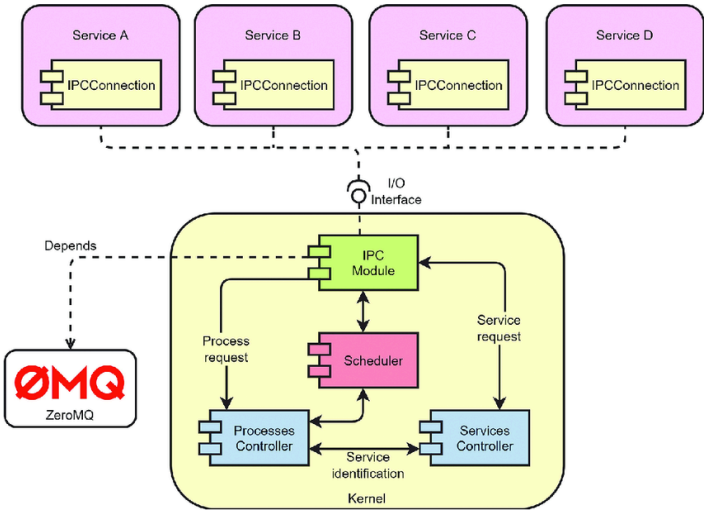
2. Memory Management and Protection: Each program needs memory to function. The kernel allocates memory dynamically, ensuring that one application doesn't intrude into another's memory space. If a program tries to access memory, it shouldn't—like writing to a null or invalid pointer in C—the kernel immediately blocks it, often resulting in a segmentation fault. This memory protection is crucial in preventing system crashes or corrupted data.

3. File and Device Management: When applications want to store files or communicate with hardware like SSDs, GPUs, or network adapters, they must go through the kernel. The kernel provides file system access and manages device drivers, enabling communication with hardware while maintaining system stability. Even simple actions in Python, like writing to a file, trigger a kernel-handled system call.

4. Interrupt Handling: Hardware often needs to grab the system's attention, like a keypress or incoming network data. This is done through interrupts, and the kernel is always listening. When an interrupt occurs, the kernel temporarily halts the current task, processes the interrupt, and resumes normal execution. This responsiveness is crucial for real-time performance.

> ⚠️ If the kernel fails, the **entire system crashes**. There's no backup process because the kernel itself is the backbone of system operations. In Windows, this results in the infamous **Blue Screen of Death**; in Linux or macOS, it's known as a **kernel panic**. A particularly dangerous moment in kernel history occurred in 2018 with the discovery of the **Meltdown** vulnerability. This hardware-level security bug allowed unauthorized access to kernel memory from user-space applications—an event that shook the tech industry and required emergency patches across all major OS platforms.

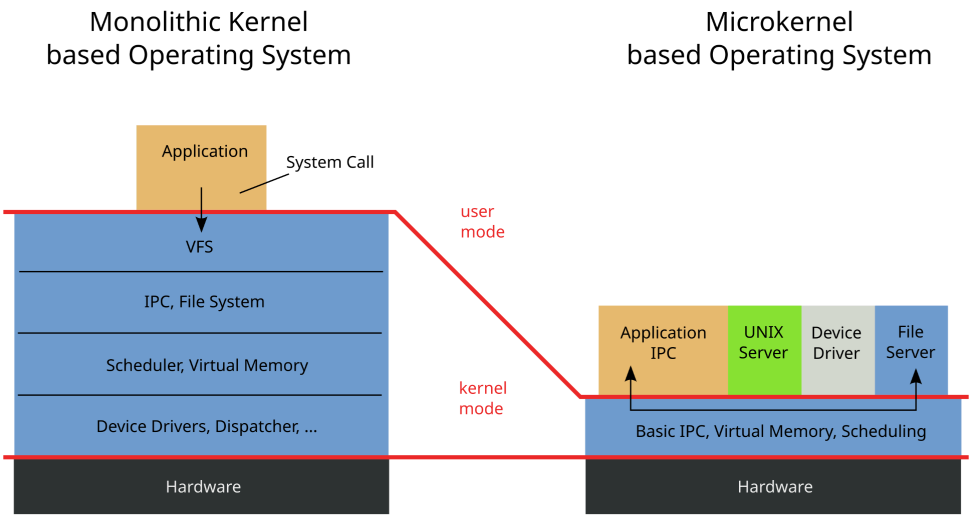
Why Engineers Obsess over Kernel Design



Kernel design sits at the crossroads of performance, security, hardware efficiency, and stability.

Engineers care deeply about how kernels are structured because they affect system responsiveness, scalability, and fault tolerance. Whether optimizing I/O operations, tuning thread scheduling, or debugging system crashes, understanding the kernel is essential. For high-performance computing environments or safety-critical applications (e.g., automotive or aerospace systems), a well-designed kernel can be the difference between reliable execution and catastrophic failure.

Monolithic vs Microkernel- Tradeoffs



🧩 Monolithic Kernels

In a monolithic kernel, almost all core services—including device drivers, file systems, memory management, and system calls—run in kernel space. This design offers high performance and speed, as everything communicates internally without the need for inter-process communication (IPC). Linux and Windows are primarily monolithic, although they support dynamic loading of kernel modules like drivers and system extensions.

However, monolithic kernels come with greater risk. If any component in the kernel crashes (e.g., a buggy device driver), it can take down the entire system.

🧪 Microkernels

A microkernel, by contrast, includes only the most essential services, like CPU scheduling and memory handling, inside the kernel. Other services, such as file systems or network drivers, run as separate processes in user space. This modular design is more fault-tolerant, as individual components can fail without crashing the whole OS. For example, macOS uses a microkernel-based architecture (XNU), although in practice it includes performance optimizations by running some components in kernel space, making it a hybrid kernel.

In summary, monolithic kernels are typically faster and more efficient, while microkernels are safer and more modular. Many modern operating systems blend these philosophies to get the best of both worlds.

Special Kernels

The concept of a kernel extends far beyond traditional operating systems and is used in specialized computing domains like AI, GPUs, and quantum computing.

1. In GPU Programming

In environments like CUDA or OpenCL, a "kernel" refers to a small function executed in parallel across thousands of GPU threads. These kernels perform computations on massive datasets (like tensors in AI) and are optimized for

throughput and speed. The term "kernel" here still reflects its low-level proximity to hardware and high-efficiency execution.

2. In Artificial Intelligence

Kernels in AI, especially in classical machine learning algorithms like Support Vector Machines (SVMs), refer to functions that compute similarity in high-dimensional spaces. These "kernel tricks" enable algorithms to detect complex, non-linear patterns.

3. In Quantum Computing

Emerging quantum operating systems use the term quantum kernels to describe core logic that manages how qubits(quantum bits) execute instructions. Researchers are building early-stage abstractions that resemble classical kernels to manage quantum operations efficiently. These kernels are becoming vital as quantum computing evolves toward more practical applications.

Whether built as a monolithic or microkernel, its design directly impacts how secure, fast, and scalable an OS is. And its influence now reaches into parallel computing, artificial intelligence, and even quantum physics. Though invisible to end users, the kernel is the powerhouse silently orchestrating the digital world.

Chapter 2

The Origin Story

Born Out of Necessity

In 2011, the Linux kernel was everywhere — powering smartphones, supercomputers, and even Mars helicopters. It was stable, battle-tested, and… a bit boring.

Its very success created a paradox: as Linux became the foundation of global infrastructure, bold innovations at its core became riskier. Kernel changes were rare. Disruption had to be surgical.

Meanwhile, networking was undergoing its own quiet revolution. Hardware-defined networks were giving way to software-defined networking (SDN), enabling dynamic connections without the need for physical cables. In this climate of change, a radical question began to emerge inside companies like Red Hat, Facebook, and a small, ambitious startup named PLUMgrid:

> *“What if the kernel itself could be programmable?”*

At the heart of PLUMgrid was a young kernel engineer, **Alexei Starovoitov,** who took this question seriously. He imagined a world where dynamic, safe programs could run inside the kernel, without compromising its security or stability. His first attempt failed. After just a few hours of traffic, the network stack crashed.

But from failure came clarity. Alexei realized:

> *"The kernel cannot trust user space."*

This insight became foundational. He began building a *verifier* — a system that could inspect code before it ever touched the kernel. That single idea would eventually become one of the defining pillars of eBPF.

Engineering a Revolution, Brick by Brick

To make his vision real, Alexei created a new instruction set. It was loosely inspired by the old **Berkeley Packet Filter** (BPF), but far more powerful and flexible. At the suggestion of kernel maintainer **David Miller**, Alexei renamed it: extended BPF (eBPF).

But getting it merged into the mainline Linux kernel was anything but easy. Alexei submitted a massive "patch bomb" — complete with a custom LLVM backend. It was met with resistance.

What he needed now was allies.

One of the earliest was **Daniel Borkmann** from Red Hat, who recognized eBPF's potential and joined the effort. Around the same time, **Brendan Gregg** at Netflix saw something else: eBPF could unify Linux's fragmented observability tools.

At a whiteboard session inside Netflix HQ, Brendan and Alexei made a pact:

> *"If Alexei added support for kprobes, Brendan would build the tooling to make it usable."*

That handshake wasn't just a collaboration. It was the beginning of a reframing: eBPF wasn't just a networking feature anymore — it was becoming a universal observability engine.

Breaking Into the World

In 2014, after years of effort, the **first eBPF patch was quietly accepted into the kernel**. The acceptance note? Just one line:

> *"Applied, thanks."*

Behind that casual message was a celebration of persistence.

The real momentum came soon after, when Facebook deployed eBPF at scale. Their team built **Katran**, a layer 4 load balancer that processed over 15 million packets per second with just 50 nanoseconds per packet. That's a 10x performance boost.

Then came the community moment.

At **DockerCon 2017**, eBPF took center stage. Developers like **Brendan Gregg**, **Thomas Graf**, and **Liz Rice** showcased its real-world power, from packet filtering and observability to performance tracing. For the first time, the cloud-native world saw eBPF not as an exotic kernel tool, but as a powerful *platform*.

Unlocking Security, Scaling Adoption

As eBPF proved its capabilities in networking and observability, a new frontier emerged: security.

At Google, **KP Singh** used eBPF to build the BPF **Linux Security Module** (LSM) — a flexible security system that added fine-grained hooks into the kernel. Suddenly, the old trade-off was shattered:

> *"Security didn't have to be slow anymore."*

In parallel, a new wave of adoption surged. A project called **Cilium**, built on top of eBPF, brought modern networking to Kubernetes clusters. Its creators — including **Daniel Borkmann** and **Thomas Graf** — went on to launch **Isovalent**, the company behind Cilium's continued development.

Then came a defining milestone:
Google Cloud integrated Cilium and eBPF into its GKE Autopilot platform.

From a niche kernel technology, eBPF has become a default part of enterprise infrastructure.

Beyond Linux — A Cross-Platform Movement

As adoption grew, so did ambition. Why should eBPF stop at Linux?

At Microsoft, a team led by **Dave Thaler** launched **eBPF for Windows**, building a portable runtime to bring eBPF to new environments. The logic was clear:

> *"Why reimplement the same features across platforms when one portable framework could unify them all?"*

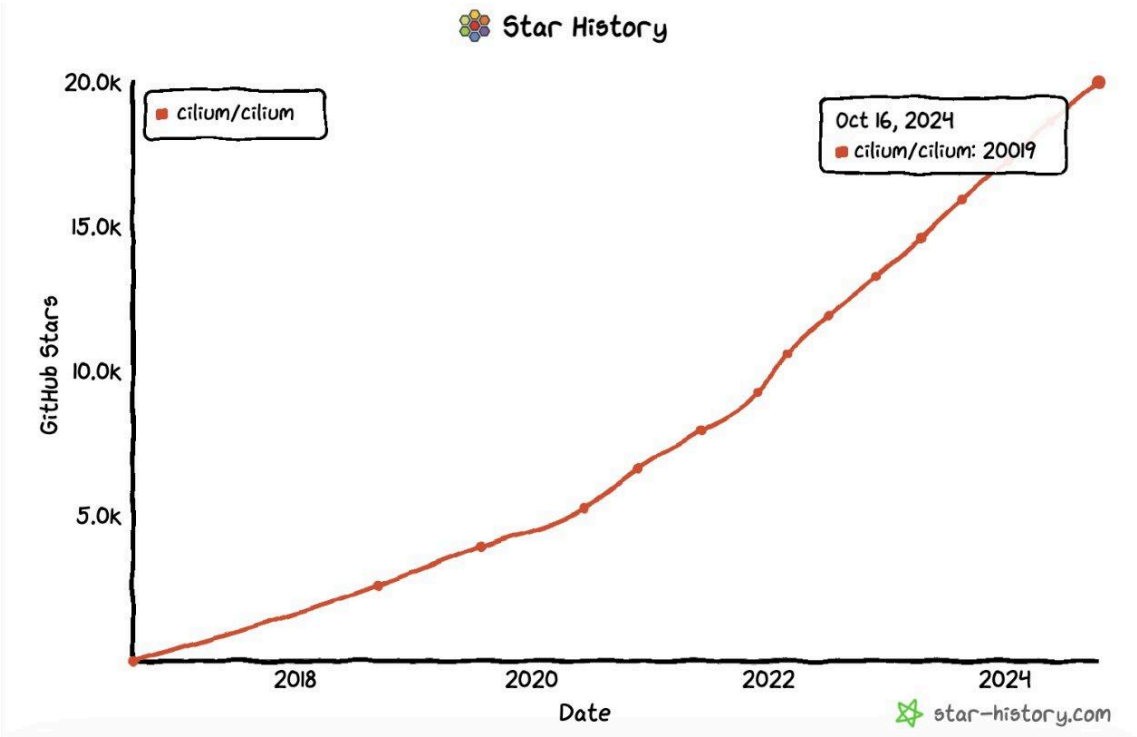
In 2021, the **eBPF Foundation** was formed, backed by Google, Netflix, Microsoft, Isovalent, and others. It became a neutral hub for cross-industry innovation. Soon, BSD and Apple began exploring eBPF as well.

eBPF was no longer just a Linux story.
It had become a global systems movement.

The Superpower of Today

By 2022, Intel was using eBPF to optimize performance across CPUs, GPUs, and AI accelerators. As **Brendan Gregg** put it,

> *"What used to take weeks, we can now do in an hour."*



eBPF has evolved into a superpower, delivering unmatched visibility, control, and performance. It now ran on every Android phone, powered top-tier security tools, and quietly underpinned the modern cloud-native stack.

What began as a radical kernel patch had become a universal runtime inside the operating system.

From custom bytecode to a revolution in systems design, eBPF didn’t just extend Linux — it transformed how we think about computing itself.

Chapter 3

What is eBPF really?

The Problem eBPF Solves

Modern computing infrastructure faces unprecedented challenges. Cloud-native applications demand real-time insights into system behavior, whether for debugging complex microservices interactions, optimizing network performance, or implementing sophisticated security policies. Traditional approaches to kernel extension—writing kernel modules—are fraught with risks: a single bug can crash the entire system, security vulnerabilities can compromise the host, and the development cycle is complex and time-consuming.

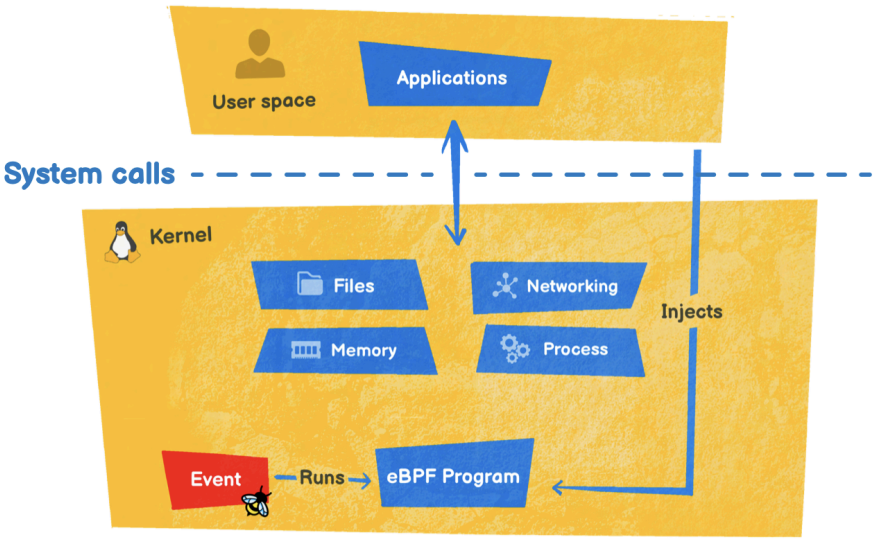
eBPF is a revolutionary technology with origins in the Linux kernel that can run sandboxed programs in a privileged context, such as the operating system kernel. It represents a paradigm shift from static kernel functionality to a programmable, extensible platform that maintains safety and performance.

The evolution from Berkeley Packet Filter (BPF) to extended BPF (eBPF) mirrors the broader transformation of computing infrastructure. Where the original BPF was designed for packet filtering, eBPF has become what many consider a "superpower" for Linux systems, enabling unprecedented visibility and control over kernel operations.

eBPF Architecture and Fundamentals

At its core, eBPF operates as a virtual machine embedded within the Linux kernel. This architecture provides several critical components that work together to enable safe, efficient kernel programming:

The eBPF Virtual Machine: The heart of eBPF is its virtual machine, which executes bytecode compiled from higher-level languages like C. Unlike traditional kernel modules that run with full privileges, eBPF programs operate within a sandboxed environment with strict safety guarantees.



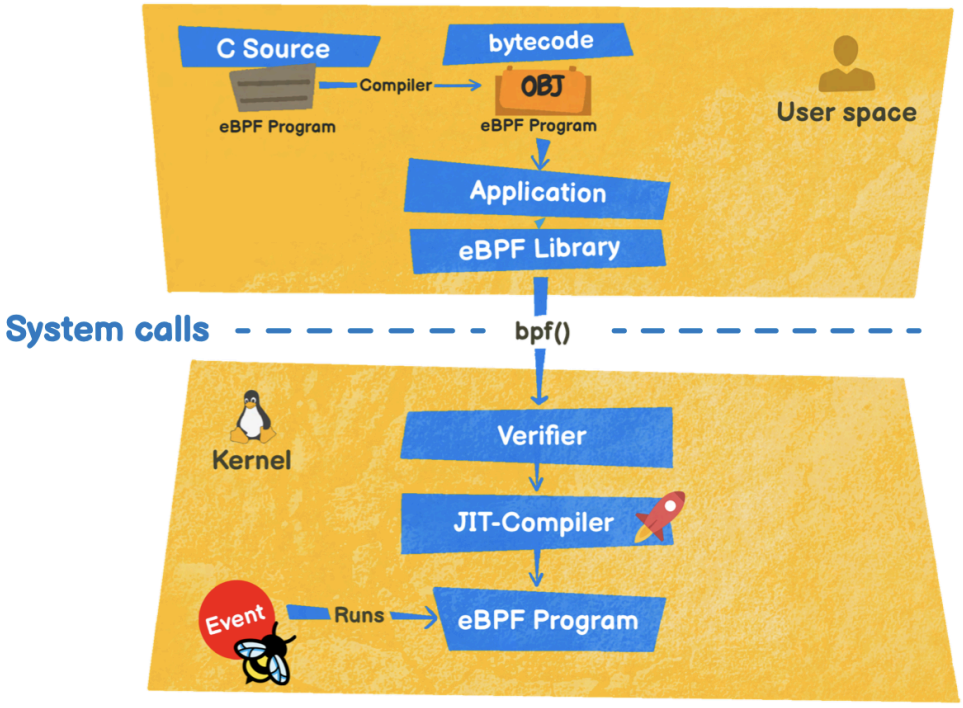
Verification Engine: Before any eBPF program executes, it must pass through the kernel's verifier. This component performs static analysis to ensure the program cannot crash the system, access unauthorized memory, or create infinite loops. The verifier is eBPF's key innovation—it makes kernel programming safe without sacrificing performance.

Maps and Data Structures: eBPF maps provide efficient mechanisms for sharing data between kernel and user space, storing state across program invocations, and communicating between different eBPF programs. These data structures are optimized for high-performance operations and include hash maps, arrays, ring buffers, and specialized structures for networking and tracing.

Helper Functions: To interact with kernel internals safely, eBPF programs use helper functions—a stable API that abstracts kernel complexity while maintaining security boundaries. These functions enable everything from packet manipulation to performance counter access.

Hook Points: eBPF programs attach to specific points in the kernel called hooks. These include network interfaces, system call entry/exit points, kernel function entry/exit (kprobes), tracepoints, and many others. Each hook type provides different capabilities and access to different kernel data structures.

The eBPF Programming Model



Programming with eBPF follows a unique model that balances power with safety. Programs are typically written in restricted C, compiled to eBPF bytecode using LLVM, and then loaded into the kernel where they're verified and executed.

Restricted C Environment: eBPF C programming operates under specific constraints. Programs cannot contain unbounded loops, must have a finite call depth, and can only access memory through approved patterns. These restrictions enable the verifier to guarantee program safety.

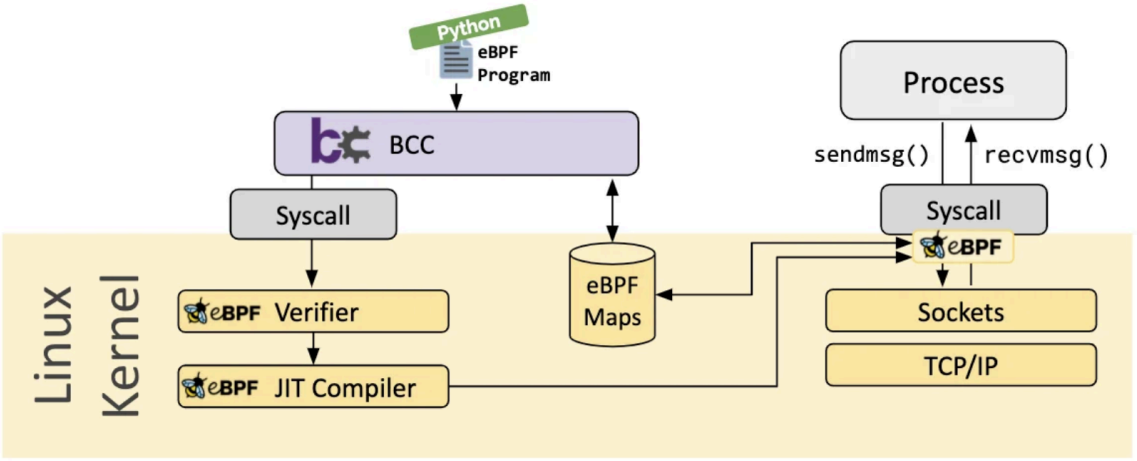
Compilation and Loading: The development workflow involves compiling C code to eBPF bytecode using tools like Clang/LLVM, then loading the bytecode into the kernel using system calls or higher-level libraries. Modern tools like libbpf provide sophisticated loading and management capabilities.

Event-Driven Execution: eBPF programs are event-driven—they execute in response to kernel events like network packet arrival, system call invocation, or timer expiration. This model is highly efficient as programs only run when relevant events occur.

Your First eBPF Experience

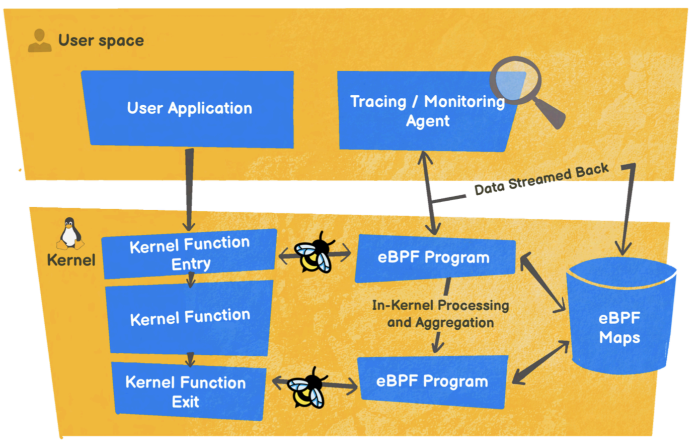
The quickest way to experience eBPF's power is through bpftrace, which provides a high-level scripting interface for eBPF programs. Consider this simple example:

This one-liner tracks every program execution on the system, demonstrating eBPF's ability to provide deep system visibility with minimal overhead. The program attaches to the execve system call tracepoint and prints the process name and executed command.



For more structured development, the BCC (BPF Compiler Collection) framework provides Python bindings and C programming templates that simplify eBPF development while maintaining full power and flexibility.

eBPF in Production Systems



Real-world eBPF deployments span multiple domains:

Observability: Tools like Pixie, Parca, and various APM solutions use eBPF to provide application-level insights without code changes. They can trace function calls, measure latency distributions, and correlate application behavior with system metrics.

Security: eBPF enables runtime security tools that can monitor and enforce policies at the kernel level. Solutions like Falco use eBPF to detect anomalous behavior, while others implement zero-trust networking models.

Networking: Beyond Cilium, projects like Katran (Facebook's load balancer) and various CNI implementations leverage eBPF for high-performance packet processing.

Performance Optimization: eBPF programs can implement caching layers, optimize routing decisions, and reduce context switches by handling operations directly in kernel space.

Chapter 4

Meet Cilium: eBPF’s Real-World Superpower

The Cloud-Native Networking Challenge

The rise of Kubernetes and cloud-native architectures has fundamentally changed networking requirements. Traditional networking solutions, built for static environments with long-lived connections, struggle with the dynamic, ephemeral nature of containerized applications. Cilium is an open-source, cloud native solution for providing, securing, and observing network connectivity between workloads, fueled by the revolutionary Kernel technology eBPF.

Legacy networking approaches rely heavily on iptables rules that can number in the thousands, creating performance bottlenecks and management complexity. IP-based security models break down when containers are created and destroyed rapidly, and traditional load balancing solutions cannot keep pace with service mesh requirements.

Cilium addresses these challenges by rebuilding the networking stack from the ground up using eBPF. Instead of relying on kernel networking subsystems designed decades ago, Cilium implements networking logic directly in eBPF programs that run in kernel space.

Cilium Architecture Deep Dive

Agent-Based Architecture: It supports dynamic insertion of eBPF bytecode into the Linux kernel at various integration points such as: network IO, application sockets, and tracepoints to implement security, networking and visibility logic. Cilium operates through agents that run as DaemonSets on each Kubernetes node. These agents coordinate with the kernel's eBPF subsystem to install, manage, and update networking programs.

Identity-Based Security Model: Rather than relying on IP addresses that change frequently in dynamic environments, Cilium implements identity-based

security. Each workload receives a cryptographic identity derived from Kubernetes labels, enabling consistent policy enforcement regardless of network location.

Datapath Architecture: Cilium's datapath consists of multiple eBPF programs that handle different aspects of networking:

- **TC (Traffic Control) programs** for ingress/egress processing
- **Socket-level programs** for connection load balancing
- **XDP programs** for high-performance packet processing
- **Sockmap programs** for socket-aware load balancing

Control Plane Integration: Cilium's control plane integrates deeply with Kubernetes APIs, watching for changes to services, endpoints, network policies, and other resources. It translates these high-level constructs into efficient eBPF programs.

Core Cilium Features

High-Performance Networking: By operating in kernel space, Cilium eliminates many context switches and memory copies that plague traditional networking solutions. eBPF programs can process packets at line rate, making Cilium suitable for high-throughput environments.

Service Mesh Without Sidecars: Cilium provides diverse networking, security, and observability capabilities all in the Linux kernel by leveraging eBPF. Traditional service meshes rely on proxy sidecars that add latency and resource overhead. Cilium can implement service mesh functionality directly in the kernel, providing L7 policy enforcement and observability without sidecars.

Multi-Cluster Networking: Cilium Cluster Mesh enables secure networking across multiple Kubernetes clusters, supporting hybrid and multi-cloud deployments. It maintains the same identity-based security model across cluster boundaries.

Network Policy Enforcement: Cilium implements Kubernetes Network Policies and extends them with L7 rules, DNS-based policies, and service-aware

restrictions. Policies are compiled into efficient eBPF programs that enforce rules with minimal overhead.

Hubble: eBPF-Powered Observability

Hubble, Cilium's observability component, demonstrates eBPF's power for network visibility. It provides:

Flow Monitoring: Every network flow is observed and recorded, including metadata about source/destination identities, protocols, and policy decisions. This data enables detailed network topology mapping and traffic analysis.

Service Dependency Mapping: By observing actual network flows, Hubble can construct accurate service dependency graphs, essential for understanding complex microservices architectures.

Security Audit Trail: Policy violations, dropped packets, and security events are logged with full context, enabling forensic analysis and compliance reporting.

Performance Metrics: Latency histograms, connection rates, and throughput metrics are collected at the kernel level, providing accurate performance data without application instrumentation.

Installing and Configuring Cilium

Prerequisites and Planning: Before deploying Cilium, consider your cluster's networking requirements. Cilium can replace kube-proxy entirely or work alongside existing CNI plugins in certain configurations. Key planning considerations include:

- Whether to enable IP address management (IPAM)
- Integration with cloud provider networking
- Multi-cluster connectivity requirements
- Required security policies and compliance needs.

Installation Methods: Cilium supports multiple installation approaches.

Cilium in Different Environments

Managed Kubernetes Services: Major cloud providers offer Cilium-powered networking options. Amazon EKS supports Cilium as an add-on, Google GKE provides Cilium-based Dataplane V2, and Azure AKS offers Cilium integration. These managed offerings simplify deployment while providing enterprise support.

Bare Metal and On-Premises: Cilium excels in bare metal environments where it can leverage advanced eBPF features without cloud provider limitations. Features like XDP acceleration and BGP integration are particularly valuable in these deployments.

Edge and IoT: Cilium's efficiency makes it suitable for edge computing scenarios where resources are constrained but networking requirements remain complex.

eBPF and Cilium represent a fundamental shift in how we think about kernel programming and cloud-native networking. eBPF has resulted in a new generation of tooling that allows developers to easily diagnose problems, innovate quickly, and extend operating system functionality.

The journey from traditional networking to eBPF-powered solutions like Cilium demonstrates the power of rethinking established paradigms. By moving networking logic into the kernel and leveraging eBPF's safety guarantees, we can achieve unprecedented performance, security, and observability.

As the ecosystem continues to mature, the combination of eBPF's kernel programming capabilities and Cilium's cloud-native networking solutions will undoubtedly play a central role in the future of infrastructure software. Whether you're implementing observability solutions, securing cloud-native applications, or optimizing network performance, understanding these technologies is essential for modern infrastructure engineering.

The key to success lies in starting with practical applications, building understanding through hands-on experience, and gradually expanding into more advanced use cases. The community resources, documentation, and tools available today make this journey more accessible than ever before.

Chapter 5

Further Reading

This appendix provides a curated collection of resources to deepen your understanding of eBPF, Cilium, and related technologies. The resources are organized by learning progression and purpose to help you continue your journey beyond this book.

Essential Reading

eBPF Fundamentals

Rice, Liz. ***Learning eBPF***: *Programming the Linux Kernel for Enhanced Observability, Networking, and Security*. O'Reilly Media, 2023. A comprehensive practical guide with hands-on examples that complement the concepts covered in this book.

Gregg, Brendan. ***BPF Performance Tools***: *Linux System and Application Observability*. Addison-Wesley, 2019. The definitive guide to performance analysis using eBPF, essential for understanding advanced observability techniques.

Calavera, David, and Lorenzo Fontana. ***Linux Observability with BPF***: *Advanced Programming for Performance Analysis and Networking*. O'Reilly Media, 2019. Deep technical exploration of eBPF's observability capabilities with practical implementations.

Systems and Performance Foundations

Gregg, Brendan. ***Systems Performance***: *Enterprise and the Cloud*. 2nd ed. Pearson, 2020. Foundational performance analysis concepts that underpin effective eBPF usage.

Love, Robert. ***Linux System Programming***: *Talking Directly to the Kernel and C Library*. 2nd ed. O'Reilly Media, 2013. Essential background for understanding the Linux programming interface that eBPF leverages.

Kerrisk, Michael. ***The Linux Programming Interface***: *A Linux and UNIX System Programming Handbook*. No Starch Press, 2010. Comprehensive reference for Linux system calls and kernel interfaces.

Official Documentation

Linux Kernel BPF Documentation https://docs.kernel.org/bpf/ The authoritative source for eBPF kernel implementation details, updated with each kernel release.

Cilium Documentation https://docs.cilium.io Complete reference for Cilium installation, configuration, and advanced features.

eBPF Foundation https://ebpf.io Central hub for eBPF specifications, tools, and community resources.

Development Resources

Libraries and Tools

libbpf - https://github.com/libbpf/libbpf Modern C library for eBPF development, representing current best practices.

BCC (BPF Compiler Collection) - https://github.com/iovisor/bcc Python and Lua frontends for rapid eBPF prototyping and development.

bpftrace - https://github.com/iovisor/bpftrace High-level tracing language for one-liner eBPF programs and quick analysis.

Reference Implementations

Cilium Project - https://github.com/cilium/cilium Production-grade networking, observability, and security implementation using eBPF.

Falco - https://github.com/falcosecurity/falco Runtime security monitoring using eBPF for threat detection and compliance.

Katran - https://github.com/facebookincubator/katran Facebook's layer 4 load balancer leveraging eBPF for high-performance packet processing.

Learning Platforms

Interactive Labs

Cilium Labs - https://labs.isovalent.com Hands-on tutorials covering Cilium deployment, configuration, and troubleshooting scenarios.

Killercoda eBPF Scenarios - https://killercoda.com/ebpf Interactive browser-based eBPF programming exercises and examples.

Video Learning

eBPF Summit Recordings - https://ebpf.io/summit-2023/ Annual conference presentations covering latest eBPF developments and use cases.

Cilium Community Meetings - https://www.youtube.com/c/CiliumProject Regular technical discussions and feature demonstrations from the Cilium community.

Professional Development

Training Programs

Linux Foundation eBPF Training Structured curriculum covering eBPF development from basics to advanced topics.

Isovalent Cilium Certification Professional certification program focusing on Cilium deployment and operations.

Cloud Native Computing Foundation Training Broader cloud-native ecosystem training that includes eBPF and Cilium components.

Conferences and Events

eBPF Summit - Annual gathering of the eBPF community with technical presentations and workshops.

KubeCon + CloudNativeCon - Major cloud-native conferences featuring eBPF tracks and Cilium sessions.

Linux Plumbers Conference - Kernel development conference with eBPF subsystem discussions.

Community Resources

Discussion Forums

Cilium Slack - https://cilium.heroku.com/ Real-time community support with channels for users, developers, and specific topics.

eBPF Slack - https://ebpf.io/slack General eBPF community discussions covering tools, techniques, and troubleshooting.

Cilium GitHub Discussions - https://github.com/cilium/cilium/discussions Structured Q&A and feature discussions with maintainers and community members.

Technical Blogs

Isovalent Engineering Blog - https://isovalent.com/blog/ In-depth technical articles on eBPF implementation patterns and Cilium features.

Brendan Gregg's Blog - http://brendangregg.com/blog/ Performance analysis techniques and eBPF tool development insights.

Cilium Blog - https://cilium.io/blog/ Product updates, use case studies, and technical deep dives from the Cilium team.

Specialized Topics

Security Applications

Alves, Paulo, and Natália Ramos Cavalcanti. *Container Security: Fundamental Technology Concepts that Protect Containerized Applications*. O'Reilly Media, 2020. Context for container security challenges that eBPF and Cilium address.

Tracee Documentation - https://aquasecurity.github.io/tracee/ Runtime security and forensics using eBPF for threat detection and incident response.

Networking Deep Dives

Cilium Network Policy Editor - https://editor.cilium.io/ Interactive tool for understanding and creating Kubernetes network policies with Cilium.

eBPF Networking Documentation - https://docs.cilium.io/en/stable/concepts/ebpf/ Detailed explanation of eBPF's role in modern networking and packet processing.

Performance Analysis

eBPF Performance Tools Repository - https://github.com/brendangregg/bpf-perf-tools-book Companion repository to Brendan Gregg's book with updated tools and examples.

Linux Performance Analysis in 60 Seconds - http://brendangregg.com/Articles/Netflix_Linux_Perf_Analysis_60s.pdfQuick reference guide for performance troubleshooting using eBPF tools.

Staying Current

Release Notes and Updates

Monitor the official Cilium and Linux kernel release notes for new eBPF features and capabilities.

Research Papers

Follow academic research in systems, networking, and security conferences for cutting-edge eBPF applications.

Open Source Contributions

Consider contributing to eBPF-related projects to deepen understanding and engage with the community.

The eBPF and Cilium ecosystems evolve rapidly. Combining these resources with hands-on experimentation will provide the foundation for mastering these powerful technologies and staying current with their continued development.

Kudos to the whole eBPF community, this is all thanks to you! ❤